

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Tư, 15 tháng 10 2025, 10:27 PM
Kết thúc lúc	Thứ Tư, 15 tháng 10 2025, 11:17 PM
Thời gian thực hiện	50 phút 31 giây
Điểm	3,00/3,00
Điểm	10,00 trên 10,00 (100%)



Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement method bubbleSort() in class SLinkedList to sort this list in ascending order. After each bubble, we will print out a list to check (using printList).



```

#include <iostream>
#include <sstream>
using namespace std;

template <class T>
class SLinkedList {
public:
    class Node; // Forward declaration
protected:
    Node* head;
    Node* tail;
    int count;
public:
    SLinkedList()
    {
        this->head = nullptr;
        this->tail = nullptr;
        this->count = 0;
    }
    ~SLinkedList(){};
    void add(T e)
    {
        Node *pNew = new Node(e);

        if (this->count == 0)
        {
            this->head = this->tail = pNew;
        }
        else
        {
            this->tail->next = pNew;
            this->tail = pNew;
        }

        this->count++;
    }
    int size()
    {
        return this->count;
    }
    void printList()
    {
        stringstream ss;
        ss << "[";
        Node *ptr = head;
        while (ptr != tail)
        {
            ss << ptr->data << ",";
            ptr = ptr->next;
        }

        if (count > 0)
            ss << ptr->data << "]";
        else
            ss << "]";
        cout << ss.str() << endl;
    }
public:
    class Node {
    private:
        T data;
        Node* next;
        friend class SLinkedList<T>;
    public:
        Node() {
            next = 0;
        }
        Node(T data) {
            this->data = data;
            this->next = nullptr;
        }
    }

```

```
};

void bubbleSort();
};
```

For example:

Test	Result
int arr[] = {9, 2, 8, 4, 1};	[2,8,4,1,9]
SLinkedList<int> list;	[2,4,1,8,9]
for(int i = 0; i <int(sizeof(arr))/4;i++)	[2,1,4,8,9]
list.add(arr[i]);	[1,2,4,8,9]
list.bubbleSort();	

Answer: (penalty regime: 0 %)

Reset answer

```
1 template <class T>
2 void SLinkedList<T>::bubbleSort()
3 {
4     if (!head || !head->next) return; // Danh sách rỗng hoặc chỉ 1 phần tử
5
6     for (int i = 0; i < count - 1; i++) {
7         Node* ptr = head;
8         Node* nextNode = nullptr;
9         bool swapped = false;
10
11         for (int j = 0; j < count - i - 1; j++) {
12             nextNode = ptr->next;
13             if (ptr->data > nextNode->data) {
14                 // Hoán đổi dữ liệu giữa hai node
15                 T temp = ptr->data;
16                 ptr->data = nextNode->data;
17                 nextNode->data = temp;
18                 swapped = true;
19             }
20             ptr = ptr->next;
21         }
22
23         // Chỉ in nếu vòng này có hoán đổi (thực sự có "bubble")
24         if (swapped) {
25             printList();
26         } else {
27             // Nếu không có hoán đổi => đã sắp xếp xong
28             break;
29         }
30     }
31 }
32
```



	Test	Expected	Got	
✓	<pre>int arr[] = {9, 2, 8, 4, 1}; SLinkedList<int> list; for(int i = 0; i <int(sizeof(arr))/4;i++) list.add(arr[i]); list.bubbleSort();</pre>	<pre>[2,8,4,1,9] [2,4,1,8,9] [2,1,4,8,9] [1,2,4,8,9]</pre>	<pre>[2,8,4,1,9] [2,4,1,8,9] [2,1,4,8,9] [1,2,4,8,9]</pre>	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Implement static method `selectionSort` in class **Sorting** to sort an array in ascending order. After each selection, we will print out a list to check (using `printArray`).

```
#include <iostream>
using namespace std;

template <class T>
class Sorting
{
public:
    /* Function to print an array */
    static void printArray(T *start, T *end)
    {
        int size = end - start;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << ", ";
        cout << start[size - 1];
        cout << endl;
    }

    static void selectionSort(T *start, T *end);
};
```

For example:

Test	Result
int arr[] = {9, 2, 8, 1, 0, -2}; Sorting<int>::selectionSort(&arr[0], &arr[6]);	-2, 2, 8, 1, 0, 9 -2, 0, 8, 1, 2, 9 -2, 0, 1, 8, 2, 9 -2, 0, 1, 2, 8, 9 -2, 0, 1, 2, 8, 9

Answer: (penalty regime: 0 %)

Reset answer

```
1 template <class T>
2 void Sorting<T>::selectionSort(T *start, T *end)
3 {
4     int n = end - start;
5     for (int i = 0; i < n - 1; i++)
6     {
7         // Giả sử vị trí nhỏ nhất là i
8         int minIndex = i;
9
10        // Tìm phần tử nhỏ nhất trong đoạn [i+1, n)
11        for (int j = i + 1; j < n; j++)
12        {
13            if (start[j] < start[minIndex])
14                minIndex = j;
15        }
16
17        // Hoán đổi phần tử nhỏ nhất về đầu đoạn
18        if (minIndex != i)
19        {
20            T temp = start[i];
21            start[i] = start[minIndex];
22            start[minIndex] = temp;
23        }
24
25        // In mảng sau mỗi lần chọn
26        printArray(start, end);
27    }
28 }
```



	Test	Expected	Got	
✓	<pre>int arr[] = {9, 2, 8, 1, 0, -2}; Sorting<int>::selectionSort(&arr[0], &arr[6]);</pre>	<pre>-2, 2, 8, 1, 0, 9 -2, 0, 8, 1, 2, 9 -2, 0, 1, 8, 2, 9 -2, 0, 1, 2, 8, 9 -2, 0, 1, 2, 8, 9</pre>	<pre>-2, 2, 8, 1, 0, 9 -2, 0, 8, 1, 2, 9 -2, 0, 1, 8, 2, 9 -2, 0, 1, 2, 8, 9 -2, 0, 1, 2, 8, 9</pre>	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1.00/1.00.



Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **sortSegment** and **ShellSort** in class **Sorting** to sort an array in ascending order.

```
#ifndef SORTING_H
#define SORTING_H

#include <sstream>
#include <iostream>
#include <type_traits>
using namespace std;

template <class T>
class Sorting {
private:
    static void printArray(T* start, T* end)
    {
        int size = end - start;
        for (int i = 0; i < size; i++)
            cout << start[i] << " ";
        cout << endl;
    }

public:
    // TODO: Write your code here
    static void sortSegment(T* start, T* end, int segment_idx, int cur_segment_total);
    static void ShellSort(T* start, T* end, int* num_segment_list, int num_phases);
};

#endif /* SORTING_H */
```

For example:

Test	Result
int num_segment_list[] = {1, 3, 5}; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 };	5 segments: 5 4 3 2 1 10 9 8 7 6 3 segments: 2 1 3 5 4 7 6 8 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10
Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases);	

Answer: (penalty regime: 0 %)

Reset answer

```
1 static void sortSegment(T* start, T* end, int segment_idx, int cur_segment_total){
2     int size = end - start;
3     // insertion sort cho từng segment
4     for (int i = segment_idx + cur_segment_total; i < size; i += cur_segment_total)
5     {
6         T key = start[i];
7         int j = i - cur_segment_total;
8         while (j >= 0 && start[j] > key)
9         {
10             start[j + cur_segment_total] = start[j];
11             j -= cur_segment_total;
12         }
13         start[j + cur_segment_total] = key;
14     }
15 }
16 static void ShellSort(T* start, T* end, int* num_segment_list, int num_phases){
17     // CHẠY NGƯỢC phase để khớp "Expected"
18     for (int phase = num_phases - 1; phase >= 0; --phase)
19     {
20         int num_segments = num_segment_list[phase];
21         cout << num_segments << " segments: ";
22         // Sắp từng segment
23         for (int s = 0; s < num_segments; ++s)
24             sortSegment(start, end, s * cur_segment_total,
```



```

26         sortSegment(start, end, s, num_segments);
27
28     printArray(start, end);
29 }
30 }
31
32
33

```

	Test	Expected	Got	
✓	<pre> int num_segment_list[] = {1, 3, 5}; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases); </pre>	<pre> 5 segments: 5 4 3 2 1 10 9 8 7 6 3 segments: 2 1 3 5 4 7 6 8 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	<pre> 5 segments: 5 4 3 2 1 10 9 8 7 6 3 segments: 2 1 3 5 4 7 6 8 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	✓
✓	<pre> int num_segment_list[] = { 1, 2, 6 }; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases); </pre>	<pre> 6 segments: 4 3 2 1 6 5 10 9 8 7 2 segments: 2 1 4 3 6 5 8 7 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	<pre> 6 segments: 4 3 2 1 6 5 10 9 8 7 2 segments: 2 1 4 3 6 5 8 7 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	✓
✓	<pre> int num_segment_list[] = { 1, 2, 5 }; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases); </pre>	<pre> 5 segments: 5 4 3 2 1 10 9 8 7 6 2 segments: 1 2 3 4 5 6 7 8 9 10 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	<pre> 5 segments: 5 4 3 2 1 10 9 8 7 6 2 segments: 1 2 3 4 5 6 7 8 9 10 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	✓
✓	<pre> int num_segment_list[] = { 1, 2, 3 }; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases); </pre>	<pre> 3 segments: 1 3 2 4 6 5 7 9 8 10 2 segments: 1 3 2 4 6 5 7 9 8 10 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	<pre> 3 segments: 1 3 2 4 6 5 7 9 8 10 2 segments: 1 3 2 4 6 5 7 9 8 10 1 segments: 1 2 3 4 5 6 7 8 9 10 </pre>	✓
✓	<pre> int num_segment_list[] = { 1, 5, 8, 10 }; int num_phases = 4; int array[] = { 3, 5, 7, 10 ,12, 14, 15, 13, 1, 2, 9, 6, 4, 8, 11 }; Sorting<int>::ShellSort(&array[0], &array[15], &num_segment_list[0], num_phases); </pre>	<pre> 10 segments: 3 5 4 8 11 14 15 13 1 2 9 6 7 10 12 8 segments: 1 2 4 6 7 10 12 13 3 5 9 8 11 14 15 5 segments: 1 2 4 3 5 9 8 11 6 7 10 12 13 14 15 1 segments: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 </pre>	<pre> 10 segments: 3 5 4 8 11 14 15 13 1 2 9 6 7 10 12 8 segments: 1 2 4 6 7 10 12 13 3 5 9 8 11 14 15 5 segments: 1 2 4 3 5 9 8 11 6 7 10 12 13 14 15 1 segments: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 </pre>	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1.00/1.00.

